

디자인패턴

The Bible of Computer Science

[CS지식의 정석]

version.24.12.08

해당 교안을 무단으로 복제, 업로드, 배포, 도용할 경우
저작권법 제 97조의 저작권침해에 해당하며
5년이하의 징역 또는 5천만원 이하의 벌금에 처해질 수 있습니다.

강의에 있는 설명이 교안에 전부 들어있지 않고 요약본으로 제공되는 경우는 출판권상 책과 중복되는 부분이어서 그렇게 제공되는 점 참고부탁드립니다.
또한 강의에는 책과 중복되는 내용이라도 모두 다루고 있습니다. :)

목차

디자인패턴의 소개.....	5
디자인패턴의 의의.....	5
디자인패턴의 종류.....	5
생성패턴.....	5
구조패턴.....	5
행동 패턴.....	5
라이브러리와 프레임워크의 차이.....	6
라이브러리.....	6
프레임워크.....	6
axios 라이브러리 사례.....	6
Vue.js 프레임워크 사례.....	7
싱글톤 패턴.....	8
싱글톤이 아닌 클래스.....	8
싱글톤패턴 - JS.....	8
DB 연결 모듈.....	8
MongoDB 연결 모듈.....	9
싱글톤패턴 - JAVA.....	10
싱글톤패턴의 단점.....	10
DEEP DIVE : 싱글톤 패턴을 구현하는 7가지 방법 #1.....	11
1.단순한 메서드 호출.....	11
2.synchronized.....	13
3.정적 멤버(즉시 초기화, Eager Initialization).....	14
4.정적 블록(즉시초기화, Eager Initialization).....	15
DEEP DIVE : 싱글톤 패턴을 구현하는 7가지 방법 #2.....	15

5.정적 멤버와 Lazy Holder(중첩 클래스).....	15
6.이중 확인 잠금(DCL).....	17
volatile.....	17
7.enum.....	20
팩토리패턴.....	21
실습 - JS.....	21
실습 - java.....	22
이터레이터패턴.....	23
DI와 DIP.....	24
의존한다. 라는 의미.....	25
자바 - Project 의 DI를 적용하지 않은 사례.....	25
자바 - Project 의 DI 적용한 사례.....	27
의존관계역전원칙.....	29
의존성 주입의 장점.....	29
의존성 주입의 단점.....	29
전략패턴.....	29
옵저버 패턴.....	33
프록시 패턴.....	35
실습 - 자바스크립트.....	35
MVC MVP MVVM패턴.....	37
MVC 패턴.....	37
모델.....	37
뷰.....	37
컨트롤러.....	37
MVC패턴의 장점.....	38
MVC패턴의 단점.....	38
MVP 패턴.....	38
MVVM 패턴.....	38
차이정리.....	39
Spring의 MVC패턴 적용사례.....	39

디스패처 서블릿의 요청 처리과정.....	40
flux패턴.....	41
flux패턴의 구조.....	42
Action.....	43
Dispatcher.....	43
Store.....	43
view.....	43
flux 패턴의 장점.....	43
flux패턴이 적용된 redux.....	44
Q. 전략패턴과 의존성주입의 차이는 무엇인가요?.....	45
Q. 컨텍스트란 무엇인가요?.....	46

디자인패턴의 소개

디자인 패턴이란 프로그램을 설계할 때 발생했던 문제점들을 객체 간의 상호 관계 등을 이용하여 해결할 수 있도록 하나의 ‘규약’ 형태로 만들어 놓은 것을 의미합니다. 디자인패턴은 라이브러리나 프레임워크를 만드는데 기초적인 원리가 되며 지금도 많은 라이브러리, 프레임워크가 어떠한 디자인패턴을 기반으로 만들어지고 있습니다.

Ex) 전략패턴이 들어간 paasport.js 라이브러리 <https://www.passportjs.org/>

디자인패턴의 의의

디자인패턴들을 미리 배워 놓는다면 이걸 기반으로 여러 문제를 해결하는데 있어서 영감을 받을 수도 있고 팀원들과 협업할 때 어떤 문제가 나타났을 때 “그 부분은 전략패턴”으로 하는게 어떤가요? 라고 했을 때 빠른 의사소통이 가능할 겁니다.

디자인패턴의 종류

디자인 패턴은 크게 생성패턴, 구조패턴, 행동패턴 3가지로 나뉘집니다.

생성패턴

생성패턴은 객체 생성 방법이 들어간 디자인패턴입니다.

싱글톤, 팩토리, 추상팩토리, 빌더, 프로토타입패턴이 있습니다.

구조패턴

구조패턴은 객체, 클래스 등으로 큰 구조를 만들 때 유연하고 효율적으로 만드는 방법이 들어간 디자인패턴입니다.

프록시, 어댑터, 브리지, 복합체, 데코레이터, 퍼사드, 플라이웨이트패턴이 있습니다.

행동 패턴

객체나 클래스 간의 알고리즘, 책임 할당에 관한 디자인패턴입니다.

이터레이터, 옵저버, 전략, 책임연쇄, 커맨드, 중재자, 메멘토, 상태, 템플릿메서드, 비지터 패턴이 있습니다

이외에도 flux패턴, MVC패턴, MVVM패턴 등 수많은 디자인패턴이 있으며 이 강의에서는 중요한 몇가지의 디자인패턴을 배우게 됩니다.

라이브러리와 프레임워크의 차이

- JS코드실행사이트 : <https://playcode.io/new>
- JAVA코드 실행사이트 : <https://www.jdoodle.com/online-java-compiler/>

라이브러리

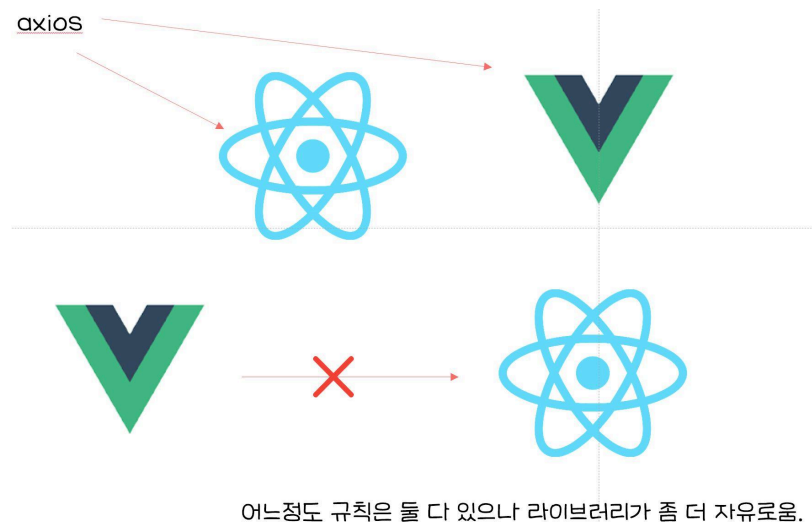
라이브러리는 공통으로 사용될 수 있는 특정한 기능들을 모듈화한 것이며 폴더명, 파일명 등에 대한 규칙이 없고 프레임워크에 비해 자유롭습니다.

ex) 차, axios

프레임워크

프레임워크는 공통으로 사용될 수 있는 특정한 기능들을 모듈화한 것을 말합니다. 폴더명, 파일명 등에 대한 규칙이 있으며 라이브러리에 비해 좀 더 엄격합니다.

ex) 비행기, vue.js, Django



어느정도 규칙은 둘 다 있으나 라이브러리가 좀 더 자유로움.

axios 라이브러리 사례

HTTP 요청 처리를 도와주는 axios의 사례입니다. 일정 규칙을 따라야 하나 프레임워크보다 자유롭고 다른 라이브러리 등에 넣을 수 있습니다.

```
axios.get('/user?ID=12345')
  .then(function (response) {
    // handle success
    console.log(response);
  })
  .catch(function (error) {
```

```

        // handle error
        console.log(error);
    })
    .then(function () {
        // always executed
    });

```

axios로 get요청을 하려면 .get()그리고 그 이후에 콜백함수 처리는 .then(callback) 등으로 해야 한다는 일정한 규칙이 있습니다. 이에 따라 해당 핸들러 함수를 넣어주어서 HTTP요청을 하고 그 요청에 따른 응답에 대한 로직을 구현하면 됩니다.

Vue.js 프레임워크 사례

Vue.js는 프론트엔드에서 많이 쓰이는 프레임워크입니다. 규칙이 있고 라이브러리보단 좀 더 엄격하며 라이브러리보다는 더 많은 기능을 제공합니다.

```

<script>
  export default {
    data() {
      return {
        count: 0
      }
    }
  }
</script>

<template>
  <button @click="count++">Count is: {{ count }}</button>
</template>

<style scoped>
  button {
    font-weight: bold;
  }
</style>

```

앞의 코드를 보면 count라는 변수를 ++하기 위해서는 data(){ 메서드 안에 변수를 선언해야 한다는 규칙 등이 있습니다.

또한 style을 보면 scoped라는 것을 통해서 해당 컴포넌트기반에서만 작동되는 style 이라는 것을 정할 수 있습니다.

싱글톤 패턴

싱글톤패턴은 하나의 클래스에 오직 하나의 인스턴스만 가지는 패턴입니다.

인스턴스생성에 많은 코스트가 드는 데이터베이스 연결모듈에 많이 쓰이며 인스턴스생성을 효율적으로 한다는 장점이 있습니다. 그러나 의존성이 높아지고 TDD를 할 때 불편한 단점이 있습니다.

싱글톤이 아닌 클래스

```
class Rectangle {
  constructor(height, width) {
    this.height = height;
    this.width = width;
  }
}
const a = new Rectangle(1, 2)
const b = new Rectangle(1, 2)
console.log(a === b) // false
```

싱글톤패턴 - JS

<https://github.com/wnghdcjfe/csnote/blob/main/ch1/2.js>

```
class Singleton {
  constructor() {
    if (!Singleton.instance) {
      Singleton.instance = this
    }
    return Singleton.instance
  }
  getInstance() {
    return this
  }
}
const a = new Singleton()
const b = new Singleton()
console.log(a === b) // true
```

DB 연결 모듈

<https://github.com/wnghdcjfe/csnote/blob/main/ch1/3.js>

```
const URL = 'mongodb://localhost:27017/kundolapp'
const createConnection = url => ({url} : url)
class DB {
  constructor(url) {
    if (!DB.instance) {
      DB.instance = createConnection(url)
    }
  }
}
```

```

    }
    return DB.instance
  }
  connect() {
    return this.instance
  }
}
const a = new DB(URL)
const b = new DB(URL)
console.log(a === b) // true

```

MongoDB 연결 모듈

https://github.com/wnghdcjfe/csnote/blob/main/ch1/singleton_db.js

```

mongoose.prototype.connect = function (uri, options, callback) {
  const _mongoose = this instanceof Mongoose ? this : mongoose;
  const conn = _mongoose.connection;
  return _mongoose._promiseOrCallback(callback, cb => {
    conn.openUri(uri, options, err => {
      if (err != null) {
        return cb(err);
      }
      return cb(null, _mongoose);
    });
  });
};

// 메인 모듈
const mysql = require('mysql');
const pool = mysql.createPool({
  connectionLimit: 10,
  host: 'example.org',
  user: 'kundol',
  password: 'secret',
  database: '승철이디비'
});
pool.connect();

// 모듈 A
pool.query(query, function (error, results, fields) {
  if (error) throw error;
  console.log('The solution is: ', results[0].solution);
});

// 모듈 B
pool.query(query, function (error, results, fields) {
  if (error) throw error;
  console.log('The solution is: ', results[0].solution);
});

```

싱글톤패턴 - JAVA

<https://github.com/wnghdcjfe/csnote/blob/main/ch1/4.java>

```
class Singleton {
    private static class singleInstanceHolder {
        private static final Singleton INSTANCE = new Singleton();
    }
    public static Singleton getInstance() {
        return singleInstanceHolder.INSTANCE;
    }
}
```

```
public class HelloWorld{
    public static void main(String []args){
        Singleton a = Singleton.getInstance();
        Singleton b = Singleton.getInstance();
        System.out.println(a.hashCode());
        System.out.println(b.hashCode());
        if (a == b){
            System.out.println(true);
        }
    }
}
```

```
/*
```

```
705927765
```

```
705927765
```

```
true
```

1. 클래스안에 클래스(*Holder*), *static*이며 중첩된 클래스인 *singleInstanceHolder*를 기반으로 객체를 선언했기 때문에 한 번만 로드되므로 싱글톤 클래스의 인스턴스는 애플리케이션 당 하나만 존재하며 클래스가 두 번 로드되지 않기 때문에 두 스레드가 동일한 *JVM*에서 2개의 인스턴스를 생성할 수 없습니다.

그렇기 때문에 동기화, 즉 *synchronized*를 신경쓰지 않아도 됩니다.

2. *final* 키워드를 통해서 *read only* 즉, 다시 값이 할당되지 않도록 했습니다.

3. 중첩클래스 *Holder*로 만들었기 때문에 싱글톤 클래스가 로드될 때 클래스가 메모리에 로드되지 않고

어떠한 모듈에서 *getInstance()*에서드가 호출할 때 싱글톤 객체를 최초로 생성 및 리턴하게 됩니다.

```
*/
```

싱글톤패턴의 단점

```
// npm install -g mocha
// mocha single1.js
const assert = require('assert');
const a = [1, 2, 3]
describe('Array', function () {
    describe('#indexOf()', function () {
        it('should return -1 when the value is not present', function () {
```

```

    assert.equal(a.indexOf(4), -1);
    a[0] = 4;
  });
});

describe('#indexOf()', function () {
  it('should return -1 when the value is not present', function () {
    assert.equal(a.indexOf(4), -1);
  });
});
});

```

DEEP DIVE : 싱글톤 패턴을 구현하는 7가지 방법 #1

면접에서 디자인패턴이 나온다면 개념 정도를 물어보지 보통 구현은 잘 나오지 않습니다.

그러나 JAVA 개발자라면 싱글톤 패턴을 구현하는 7가지 방법을 배우면서 배워가는 점들이 많기 때문에 준비했습니다.

자바를 사용하지 않은 개발자라도 참고로 배우시면 좋습니다.

코드 : <https://github.com/wnghdcjfe/csnote/tree/main/java-singleton>

1.단순한 메서드 호출

싱글톤 패턴 생성 여부를 확인하고 싱글톤이 없으면 새로 만들고 있다면 만들어진 인스턴스를 반환합니다.

그러나 이 코드는 메서드의 원자성이 결여되었습니다. 멀티스레드 환경에서는 싱글톤 인스턴스를 2개 이상 만들 수 있습니다.

1.java

```

public class Singleton {
    private static Singleton instance;

    private Singleton() {

    }

    public static Singleton getInstance() {
        if (instance == null) {
            instance = new Singleton();
        }
        return instance;
    }
}

```

```
}  
}
```

thread.java

```
public class YunhaSync {  
    private static String yunha = "오르트구름";  
  
    public static void main(String[] args) {  
        YunhaSync a = new YunhaSync();  
        new Thread(() -> {  
            for (int i = 0; i < 10; i++) {  
                a.say("사건의 지평선");  
            }  
        }).start();  
  
        new Thread(() -> {  
            for (int i = 0; i < 10; i++) {  
                a.say("오르트 구름");  
            }  
        }).start();  
  
    }  
  
    public void say(String song) {  
        yunha = song;  
        try {  
            long sleep = (long) (Math.random() * 100);  
            Thread.sleep(sleep);  
        } catch (InterruptedException e) {  
            e.printStackTrace();  
        }  
        if (!yunha.equals(song)) {  
            System.out.println(song + " | " + yunha);  
        }  
    }  
}
```

thread2.java

```
public class YunhaSync {  
    private static String yunha = "오르트구름";
```

```

public static void main(String[] args) {
    YunhaSync a = new YunhaSync();
    new Thread(() -> {
        for (int i = 0; i < 10; i++) {
            a.say("사건의 지평선");
        }
    }).start();

    new Thread(() -> {
        for (int i = 0; i < 10; i++) {
            a.say("오르트 구름");
        }
    }).start();
}

public synchronized void say(String song) {
    yunha = song;
    try {
        long sleep = (long) (Math.random() * 100);
        Thread.sleep(sleep);
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
    if (!yunha.equals(song)) {
        System.out.println(song + " | " + yunha);
    }
}
}

```

2.synchronized

인스턴스를 반환하기 전까지 격리 공간에 놓기 위해 synchronized 키워드로 잠금을 할 수 있습니다. 최초로 접근한 스레드가 해당 메서드 호출시에 다른 스레드가 접근하지 못하도록 잠금(lock)을 걸어줍니다.

이 때 getInstance() 메서드를 호출할 때마다 lock이 걸려 성능저하가 됩니다.

인스턴스가 만들어졌는데도 getInstance()는 호출이 가능하니 해당부분도 좋지 않죠.

2.java

```

public class Singleton {

    private static Singleton instance;

    private Singleton() {
    }

    public static synchronized Singleton getInstance() {
        if (instance == null) {
            instance = new Singleton();
        }
        return instance;
    }
}

```

3.정적 멤버(즉시 초기화, Eager Initialization)

정적(static) 멤버로 싱글톤을 만드는 방법입니다. Singleton 클래스가 참조될 때 로딩과 초기화가 이루어는데 이 때 메모리 상에서 한 번만 생성되고 프로그램 종료 시까지 유지됩니다.

3.java

```

public class Singleton {

    private final static Singleton instance = new Singleton();

    private Singleton() {
    }

    public static Singleton getInstance() {
        return instance;
    }
}

```

하지만 앞의 코드의 경우 getInstance()를 호출하지 않더라도 Singleton 클래스가 로드되기만 하면 메모리에 즉시 할당하는 문제점이 있습니다.

4.정적 블록(즉시초기화, Eager Initialization)

정적(static) 블록을 사용해도 됩니다.

4.java

```
public class Singleton {  
  
    private static Singleton instance = null;  
  
    static {  
        instance = new Singleton();  
    }  
  
    private Singleton() {  
  
    }  
  
    public static Singleton getInstance() {  
        return instance;  
    }  
}
```

DEEP DIVE : 싱글톤 패턴을 구현하는 7가지 방법 #2

5.정적 멤버와 Lazy Holder(중첩 클래스)

singletonHolder라는 내부클래스를 하나 더 만듦으로써, Singleton클래스가 최초에 로딩되더라도 함께 초기화가 되지 않고. getInstance()가 호출될 때 singletonHolder 클래스가 로딩되어 인스턴스를 생성하게 됩니다.

5.java

```
class Singleton {  
    private static class singletonHolder {  
        private static final Singleton INSTANCE = new Singleton();  
    }  
    public static Singleton getInstance() {  
        return singletonHolder.INSTANCE;  
    }  
}
```

앞의 코드를 좀더 업그레이드한 코드는 다음과 같습니다.

다음의 코드는 private 생성자로 클래스 외부에서 인스턴스 생성을 방지할 수 있습니다.

```
class Singleton {
    private Singleton() {} // private 생성자로 클래스 외부에서
    인스턴스 생성을 방지

    private static class singleInstanceHolder {
        private static final Singleton INSTANCE = new Singleton();
    }

    public static Singleton getInstance() {
        return singleInstanceHolder.INSTANCE;
    }
}
```

예를 들어 다음과 같이 main 클래스에서 new Singleton()으로 기존 코드는 선언할 수 있는 반면 private 생성자를 쓰게 되면 외부에서 new Singleton()으로 직접적으로 인스턴스 생성은 불가능해집니다.

즉, private 생성자를 사용함으로써 클래스의 인스턴스가 클래스 외부에서 임의로 생성되는 것을 방지하며, 싱글톤 인스턴스에 접근하기 위해서는 getInstance() 메서드를 강제하게 됩니다.

```
class Singleton {
    private Singleton() {}
    private static class singleInstanceHolder {
        private static final Singleton INSTANCE = new Singleton();
    }
    public static Singleton getInstance() {
        return singleInstanceHolder.INSTANCE;
    }
}

public class Main {
    public static void main(String[] args) {
        Singleton a1 = new Singleton(); // 해당 인스턴스 생성을
        못함.
        Singleton a2 = Singleton.getInstance();
    }
}
```

```
}
```

6. 이중 확인 잠금(DCL)

이중 확인 잠금(DCL, Double Checked Locking)도 있습니다. 인스턴스 생성 여부를 싱글톤 패턴 잠금 전에 한번, 객체를 생성하기 전에 한 번 2번 체크하면 인스턴스가 존재하지 않을 때만 잠금을 걸 수 있기 때문에 앞서 생겼던 문제점을 해결할 수 있습니다.

6.java

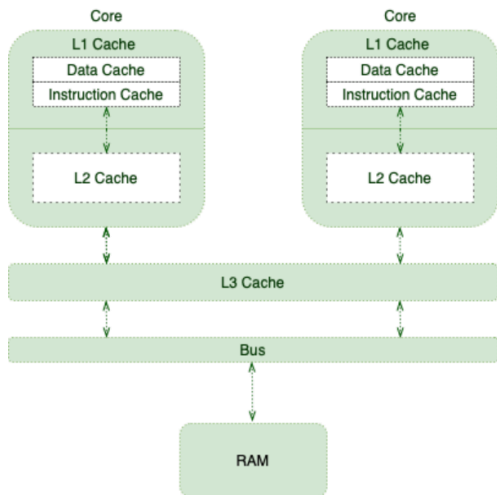
```
public class Singleton {  
  
    private volatile Singleton instance;  
  
    private Singleton() {  
  
    }  
  
    public Singleton getInstance() {  
        if (instance == null) {  
            synchronized (Singleton.class) {  
                if (instance == null) {  
                    instance = new Singleton();  
                }  
            }  
        }  
        return instance;  
    }  
}
```

volatile

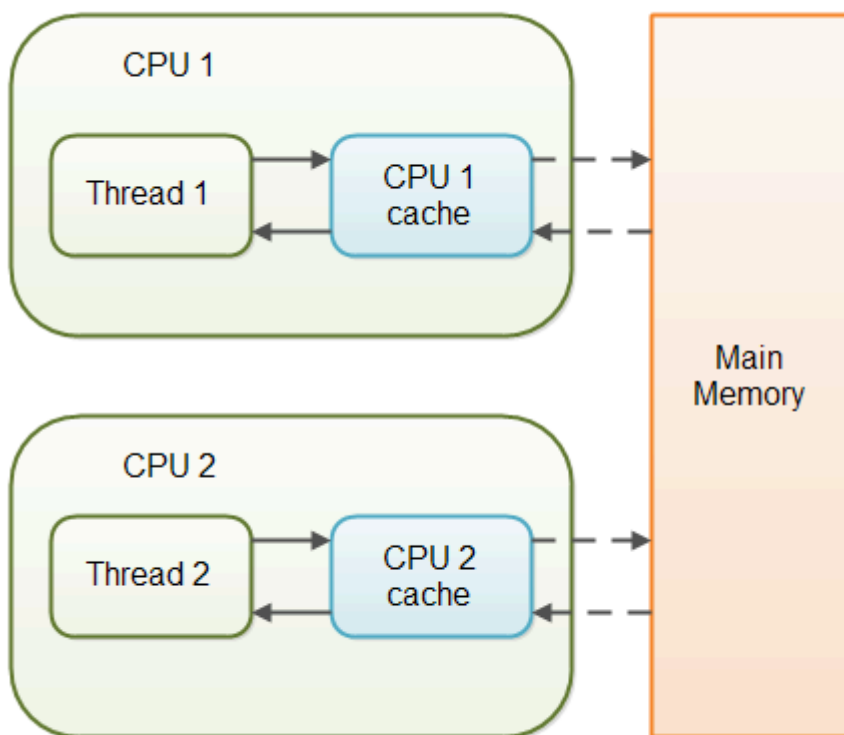
여기서 instance라는 변수에 volatile 키워드를 건 것을 볼 수 있는데요.

왜 그랬을까요?

메모리 구조는 다음과 같습니다. 메인 메모리 위에 CPU 캐시메모리라고 불리는 L3, L2, L1 캐시가 있습니다. (L4도 드물긴 하지만 L4까지 CPU 캐시 메모리라고 부릅니다.)



JAVA에서는 스레드 2개가 열리면 변수를 메인 메모리(RAM)으로부터 가져오는 것이 아니라 캐시메모리에서 각각의 캐시메모리를 기반으로 가져오게 됩니다.



volatile.java

```
public class Test {
    boolean flag = true;

    public void test() {
        new Thread(()->{
            int cnt = 0;
```

```

        while (flag) {
            cnt++;
        }
        System.out.println("Thread1 finished\n");
    }
    ).start();
    new Thread(()-> {
        try {
            Thread.sleep(100);
        } catch (InterruptedException ignored) {

        }
        System.out.println("flag to false");
        flag = false;
    }
    ).start();
}

public static void main(String[] args) {
    new Test().test();
}
}

```

그렇기 때문에 앞의 코드와 같은 변수 값 불일치 문제가 발생할 수 있죠.

근데 여기서 volatile 키워드를 추가하게 되면 Main Memory를 기반으로 저장하고 읽어오기 때문에 이 문제를 해결할 수 있습니다.

volatile2.java

```

public class Test {
    volatile boolean flag = true;

    public void test() {
        new Thread(()->{
            int cnt = 0;
            while (flag) {
                cnt++;
            }
            System.out.println("Thread1 finished\n");
        }
        ).start();
        new Thread(()-> {

```

```

        try {
            Thread.sleep(100);
        } catch (InterruptedException ignored) {

        }
        System.out.println("flag to false");
        flag = false;
    }
    ).start();
}

public static void main(String[] args) {
    new Test().test();
}
}

```

7.enum

enum의 인스턴스는 기본적으로 스레드세이프(thread safe)한 점이 보장되기 때문에 이를 통해 생성할 수 있습니다.

7.java

```

public enum SingletonEnum {
    INSTANCE;
    public void oortCloud() {

    }
}

```

그래서 최고의 방법은 무엇일까?

5번과 7번을 추천드립니다. 5번은 가장 많이 쓰인다고 알려져있고 7번은 이펙티브 자바를 쓴 조슈아 블로크가 추천한 방법입니다.

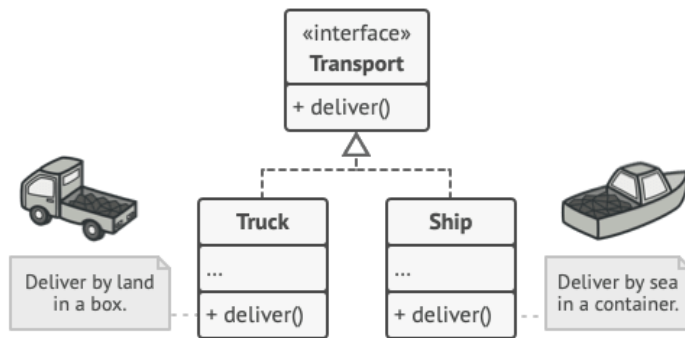
A single-element enum type is the best way to implement a singleton

- Joshua Bloch, Effective Java 2nd Edition p.18

팩토리패턴

팩토리패턴이란 상속 관계에 있는 두 클래스에서 상위 클래스가 중요한 뼈대를 결정하고, 하위 클래스에서 객체 생성에 관한 구체적인 내용을 결정하는 패턴입니다.

상위 클래스에서는 객체 생성방식에 대해 알 필요가 없어서 유연성을 갖게 되며 객체 생성 로직은 하위클래스에서만 관리 되기 때문에 유지보수성이 증가됩니다.



실습 - JS

<https://github.com/wnghdcjfe/csnote/blob/main/ch1/5.js>

```
class CoffeeFactory {
  static createCoffee(type) {
    const factory = factoryList[type]
    return factory.createCoffee()
  }
}

class Latte {
  constructor() {
    this.name = "latte"
  }
}

class Espresso {
  constructor() {
    this.name = "Espresso"
  }
}

class LatteFactory extends CoffeeFactory{
  static createCoffee() {
    return new Latte()
  }
}
```

```

}
class EspressoFactory extends CoffeeFactory{
    static createCoffee() {
        return new Espresso()
    }
}
const factoryList = { LatteFactory, EspressoFactory }

const main = () => {
    // 라떼 커피를 주문한다.
    const coffee = CoffeeFactory.createCoffee("LatteFactory")
    // 커피 이름을 부른다.
    console.log(coffee.name) // Latte
}
main()

```

실습 - java

```

enum CoffeeType {
    LATTE,
    ESPRESSO
}

abstract class Coffee {
    protected String name;

    public String getName() {
        return name;
    }
}

class Latte extends Coffee {
    public Latte() {
        name = "latte";
    }
}

class Espresso extends Coffee {
    public Espresso() {
        name = "Espresso";
    }
}

```

```

    }
}

class CoffeeFactory {
    public static Coffee createCoffee(CoffeeType type) {
        switch (type) {
            case LATTE:
                return new Latte();
            case ESPRESSO:
                return new Espresso();
            default:
                throw new IllegalArgumentException("Invalid coffee
type: " + type);
        }
    }
}

public class Main {
    public static void main(String[] args) {
        Coffee coffee =
CoffeeFactory.createCoffee(CoffeeType.LATTE);
        System.out.println(coffee.getName()); // Latte
    }
}

```

이터레이터패턴

이터레이터패턴은 이터레이터(iterator)를 사용하여 컨테이너의 요소들에 접근하는 디자인 패턴입니다. 각기 다른 자료구조들을 똑같은 인터페이스로 순회를 쉽게 할 수 있다는 장점이 있습니다. 컨테이너란 동일한 요소들을 담아놓는 집합을 말합니다. 배열, 맵 등이 있습니다.

ex) 자바스크립트의 for of 가 하나의 예

<https://github.com/wnghdcjfe/csnote/blob/main/ch1/11.js>

```

const mp = new Map()
mp.set('a', 1)
mp.set('b', 2)
mp.set('ccc', 3)
const st = new Set()

```

```

st.add(1)
st.add(2)
st.add(3)
const a = []
for(let i = 0; i < 10; i++)a.push(i)

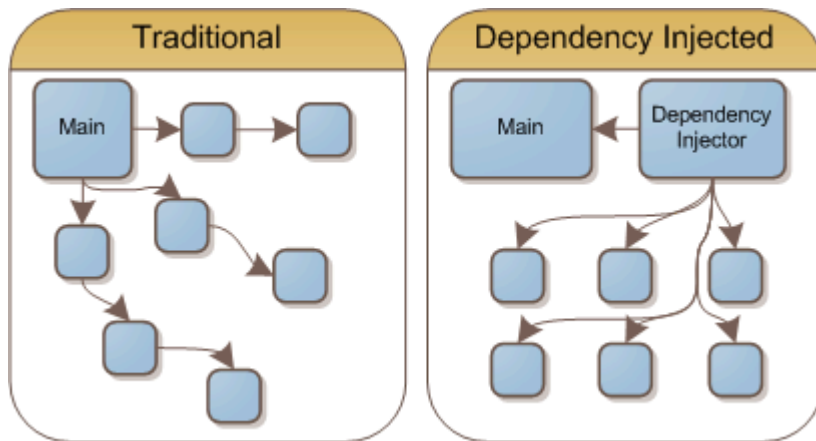
for(let aa of a) console.log(aa)
for(let a of mp) console.log(a)
for(let a of st) console.log(a)
/*
0
1
2
3
4
5
6
7
8
9
[ 'a', 1 ]
[ 'b', 2 ]
[ 'cccc', 3 ]
1
2
3

*/

```

DI와 DIP

의존성주입(DI, Dependency Injection)이란 메인 모듈(main module)이 '직접' 다른 하위 모듈에 대한 의존성을 주기보다는 중간에 의존성 주입자(dependency injector)가 이 부분을 가로채 메인 모듈이 '간접'적으로 의존성을 주입하는 방식입니다.



이를 통해 메인 모듈과 하위모듈간의 의존성을 조금 더 느슨하게 만들 수 있으며 모듈을 쉽게 교체 가능한 구조로 만듭니다.

의존한다. 라는 의미

A가 B에 의존한다. = B가 변하면 A에 영향을 미치는 관계 = A - > B
를 의미하며 코드로는 이러한 것을 A가 B에 의존한다고 합니다.

```
import java.util.*;

class B {
    public void go() {
        System.out.println("B의 go()함수");
    }
}

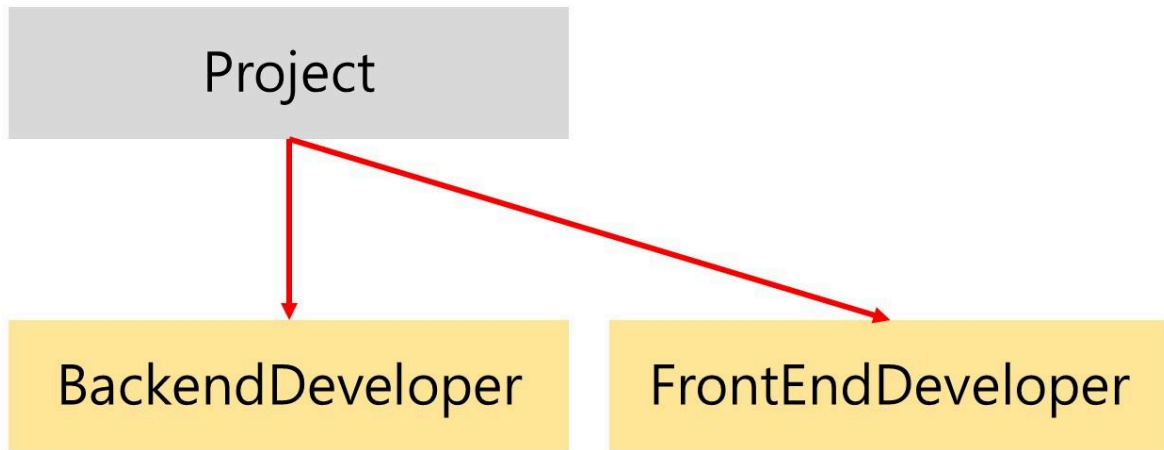
class A {
    public void go() {
        new B().go();
    }
}

public class main{
    public static void main(String args[]) {
        new A().go();
    }
}

// B의 go()함수
```

자바 - Project 의 DI를 적용하지 않은 사례

프로젝트라는 클래스를 만든다고 해보죠. 프론트엔드개발자, 백엔드 개발자가 필요할 거고 이를 다음과 같은 코드로 구현할 수 있습니다.



```
import java.util.*;

class BackendDeveloper {
    public void writeJava() {
        System.out.println("자바가 좋아 인터네셔널~");
    }
}

class FrontEndDeveloper {
    public void writeJavascript() {
        System.out.println("자바스크립트가 좋아 인터네셔널~");
    }
}

public class Project {
    private final BackendDeveloper backendDeveloper;
    private final FrontEndDeveloper frontEndDeveloper;

    public Project(BackendDeveloper backendDeveloper, FrontEndDeveloper
frontEndDeveloper) {
        this.backendDeveloper = backendDeveloper;
        this.frontEndDeveloper = frontEndDeveloper;
    }

    public void implement() {
        backendDeveloper.writeJava();
        frontEndDeveloper.writeJavascript();
    }

    public static void main(String args[]) {
        Project a = new Project(new BackendDeveloper(), new
FrontEndDeveloper());
        a.implement();
    }
}
```

[참고]

이 예시에서 Project 클래스는 BackendDeveloper와 FrontEndDeveloper 인스턴스를 생성자를 통해 전달받습니다. 이는 생성자 주입(Constructor Injection)의 한 형태로, Project 객체가 생성될 때 필요한 의존 객체들을 외부에서 받아오게 됩니다.

즉, 생성자를 주입하고 있다는 측면에서 어느정도는 DI라고 볼 수 있습니다.

그러나 Project 클래스는 BackendDeveloper와 FrontEndDeveloper 클래스의 구체적인 인스턴스에 직접적으로 의존하고 있습니다.

이는 DIP를 완전히 준수하지 않습니다. 왜냐하면, Project가 구체적인 클래스가 아닌 추상화에 의존하도록 설계되어야 하지만, 코드 상에서는 직접적인 클래스 타입에 의존하고 있기 때문입니다.

즉, DIP를 준수하지 않은 DI라고 볼 수 있습니다.

여기서 보통은 DIP를 준수하지 않은 DI를 보면서 올바르게 DI가 적용되었다 라고 하지는 않습니다.

예를 들어, 어떤 서비스 클래스가 데이터베이스 접근 인터페이스에 의존하고 있을 때, 개발 환경에서는 인메모리 데이터베이스 구현을, 운영 환경에서는 실제 SQL 데이터베이스 구현을 DI를 통해 주입할 수 있습니다. 이렇게 함으로써, 서비스 클래스는 데이터베이스 접근의 구체적인 구현 방법을 몰라도 되게 만드는 것인데 이 때 DI를 적용했다라고 하면 (DIP를 따르는) DI라고 보통은 판단합니다.

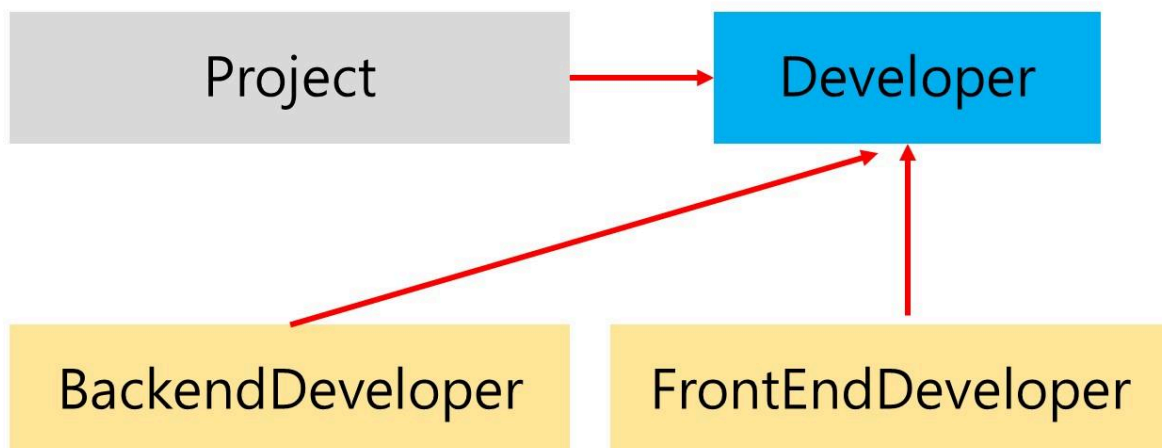
자바 - Project 의 DI 적용한 사례

DI를 적용한 모습입니다.

여러명의 개발자를 추가할 수도 있으며 또 프론트엔드개발자 말고 안드로이드 개발자 등으로 교체도 쉽게 할 수 있는 구조임을 보여줍니다.

또한 의존적인 화살표가 “역전” 된 것을 볼 수 있습니다.

DI를 하게 되면 의존관계역전원칙(Dependency Inversion Principle)이 적용되는 것이죠.



```
import java.util.*;
```

```

interface Developer {
    void develop();
}

class BackendDeveloper implements Developer {
    @Override
    public void develop() {
        writeJava();
    }

    public void writeJava() {
        System.out.println("자바가 좋아~ 새뽕새뽕");
    }
}

class FrontendDeveloper implements Developer {
    @Override
    public void develop() {
        writeJavascript();
    }

    public void writeJavascript() {
        System.out.println("자바스크립트가 좋아~ 새뽕새뽕");
    }
}

public class Project {
    private final List<Developer> developers;

    public Project(List<Developer> developers) {
        this.developers = developers;
    }

    public void implement() {
        developers.forEach(Developer::develop);
    }

    public static void main(String args[]) {
        List<Developer> dev = new ArrayList<>();
        dev.add(new BackendDeveloper());
        dev.add(new FrontendDeveloper());
        Project a = new Project(dev);
        a.implement();
    }
}

```

의존관계역전원칙

의존성 주입을 할 때는 의존관계역전원칙(DIP, Dependency Inversion Principle)이 적용됩니다.

이는 2가지의 규칙을 지키는 상태를 말합니다.

- 상위 모듈은 하위 모듈에 의존해서는 안 된다. 둘 다 추상화에 의존해야 한다.
- 추상화는 세부사항에 의존해서는 안 된다. 세부 사항은 추상화에 따라 달라져야 한다.

의존성 주입의 장점

1. 외부에서 모듈을 생성하여 `dev.add(new BackendDeveloper())` 이런식으로 집어넣는 구조가 되기 때문에 모듈들을 쉽게 교체할 수 있는 구조가 됩니다.
2. 단위 테스트와 마이그레이션이 쉬워집니다.
3. 애플리케이션 의존성 방향이 좀 더 일관되어 코드를 추론하기가 쉬워집니다.

마이그레이션 : 다른 운영환경으로 이동하는 것(DB이동, 데이터 이동 등)

의존성 주입의 단점

1. 결국에는 모듈이 더 생기게 되므로 복잡도가 증가합니다.
2. 종속성 주입자체가 컴파일을 할 때가 아닌 런타임 때 일어나기 때문에 컴파일을 할 때 종속성 주입에 관한 에러를 잡기가 어려워질 수 있습니다.

전략패턴

전략패턴이란 전략이라고 부르는 '캡슐화한 알고리즘'을 컨텍스트 안에서 바꿔주면서 상호 교체가 가능하게 만드는 디자인 패턴입니다.

<https://github.com/wnghdcjfe/csnote/blob/main/ch1/7.java>

```
import java.text.DecimalFormat;
import java.util.ArrayList;
import java.util.List;
interface PaymentStrategy {
    public void pay(int amount);
}

class KAKAOCardStrategy implements PaymentStrategy {
    private String name;
```

```

    private String cardNumber;
    private String cvv;
    private String dateOfExpiry;

    public KAKAOCardStrategy(String nm, String ccNum, String cvv,
String expiryDate){
        this.name=nm;
        this.cardNumber=ccNum;
        this.cvv=cvv;
        this.dateOfExpiry=expiryDate;
    }

    @Override
    public void pay(int amount) {
        System.out.println(amount + " paid using KAKAOCard.");
    }
}

class LUNACardStrategy implements PaymentStrategy {
    private String emailId;
    private String password;

    public LUNACardStrategy(String email, String pwd){
        this.emailId=email;
        this.password=pwd;
    }

    @Override
    public void pay(int amount) {
        System.out.println(amount + " paid using LUNACard.");
    }
}

class Item {
    private String name;
    private int price;
    public Item(String name, int cost){
        this.name=name;
        this.price=cost;
    }

    public String getName() {

```

```

        return name;
    }

    public int getPrice() {
        return price;
    }
}

class ShoppingCart {
    List<Item> items;

    public ShoppingCart(){
        this.items=new ArrayList<Item>();
    }

    public void addItem(Item item){
        this.items.add(item);
    }

    public void removeItem(Item item){
        this.items.remove(item);
    }

    public int calculateTotal(){
        int sum = 0;
        for(Item item : items){
            sum += item.getPrice();
        }
        return sum;
    }

    public void pay(PaymentStrategy paymentMethod){
        int amount = calculateTotal();
        paymentMethod.pay(amount);
    }
}

public class HelloWorld{
    public static void main(String []args){
        ShoppingCart cart = new ShoppingCart();

        Item A = new Item("kundo1A",100);
    }
}

```

```

    Item B = new Item("kundolB",300);

    cart.addItem(A);
    cart.addItem(B);

    // pay by LUNACard
    cart.pay(new LUNACardStrategy("kundol@example.com",
    "pukubababo"));
    // pay by KAKAOCBank
    cart.pay(new KAKAOCardStrategy("Ju hongchul", "123456789",
    "123", "12/01"));
  }
}
/*
400 paid using LUNACard.
400 paid using KAKAOCard.
*/

```

ex) passport.js

<https://github.com/wnghdcjfe/csnote/blob/main/ch1/8.js>

```

var passport = require('passport')
    , LocalStrategy = require('passport-local').Strategy;

passport.use(new LocalStrategy(
  function(username, password, done) {
    User.findOne({ username: username }, function (err, user)
    {
      if (err) { return done(err); }
      if (!user) {
        return done(null, false, { message: 'Incorrect
username.' });
      }
      if (!user.validPassword(password)) {
        return done(null, false, { message: 'Incorrect
password.' });
      }
      return done(null, user);
    });
  })
));

```

옵저버 패턴

옵저버패턴이란 주체가 어떤 객체(subject)의 상태 변화를 관찰하다가 상태 변화가 있을 때마다 메서드 등을 통해 옵저버 목록에 있는 옵저버들에게 변화를 알려 주는 디자인 패턴입니다.

트위터의 메인 로직, 그리고 MVC패턴에도 적용되었습니다.

<https://github.com/wnghdcjfe/csnote/blob/main/ch1/9.java>

```
import java.util.ArrayList;
import java.util.List;

interface Subject {
    public void register(Observer obj);
    public void unregister(Observer obj);
    public void notifyObservers();
    public Object getUpdate(Observer obj);
}

interface Observer {
    public void update();
}

class Topic implements Subject {
    private List<Observer> observers;
    private String message;

    public Topic() {
        this.observers = new ArrayList<>();
        this.message = "";
    }

    @Override
    public void register(Observer obj) {
        if (!observers.contains(obj)) observers.add(obj);
    }

    @Override
    public void unregister(Observer obj) {
        observers.remove(obj);
    }
}
```

```

    }

    @Override
    public void notifyObservers() {
        this.observers.forEach(Observer::update);
    }

    @Override
    public Object getUpdate(Observer obj) {
        return this.message;
    }

    public void postMessage(String msg) {
        System.out.println("Message sended to Topic: " + msg);
        this.message = msg;
        notifyObservers();
    }
}

class TopicSubscriber implements Observer {
    private String name;
    private Subject topic;

    public TopicSubscriber(String name, Subject topic) {
        this.name = name;
        this.topic = topic;
    }

    @Override
    public void update() {
        String msg = (String) topic.getUpdate(this);
        System.out.println(name + ":: got message >> " + msg);
    }
}

public class HelloWorld {
    public static void main(String[] args) {
        Topic topic = new Topic();
        Observer a = new TopicSubscriber("a", topic);
        Observer b = new TopicSubscriber("b", topic);
        Observer c = new TopicSubscriber("c", topic);
        topic.register(a);
    }
}

```

```

    topic.register(b);
    topic.register(c);

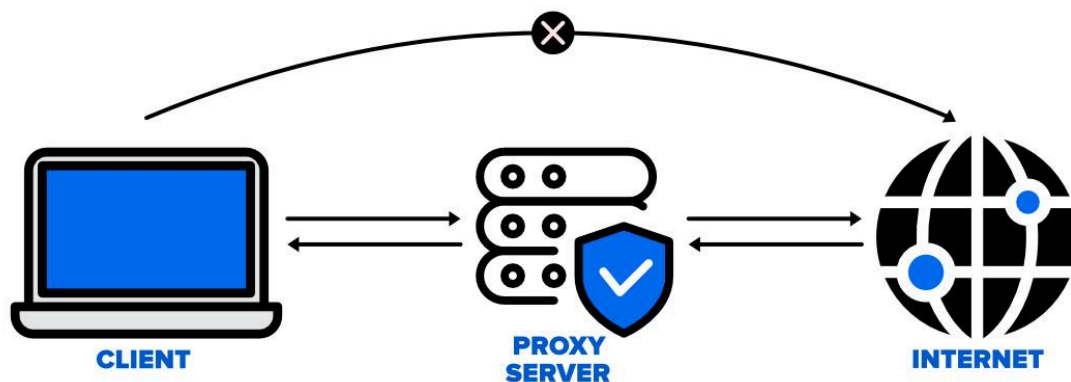
    topic.postMessage("amumu is op champion!!");
  }
}
/*
Message sended to Topic: amumu is op champion!!
a:: got message >> amumu is op champion!!
b:: got message >> amumu is op champion!!
c:: got message >> amumu is op champion!!
*/

```

프록시 패턴

프록시패턴이란 객체가 어떤 대상 객체에 접근하기 전, 그 접근에 대한 흐름을 가로채서 해당 접근을 필터링하거나 수정하는 등의 역할을 하는 계층이 있는 디자인패턴입니다.

ex) 프록시서버



서버 앞단에 두어 캐싱, 로깅 등에 활용하는 프록시서버가 대표적인 예입니다.

실습 - 자바스크립트

<https://github.com/wnghdcjfe/csnote/blob/main/ch1/10.js>

```

function createReactiveObject(target, callback) {
  const proxy = new Proxy(target, {
    set(obj, prop, value){

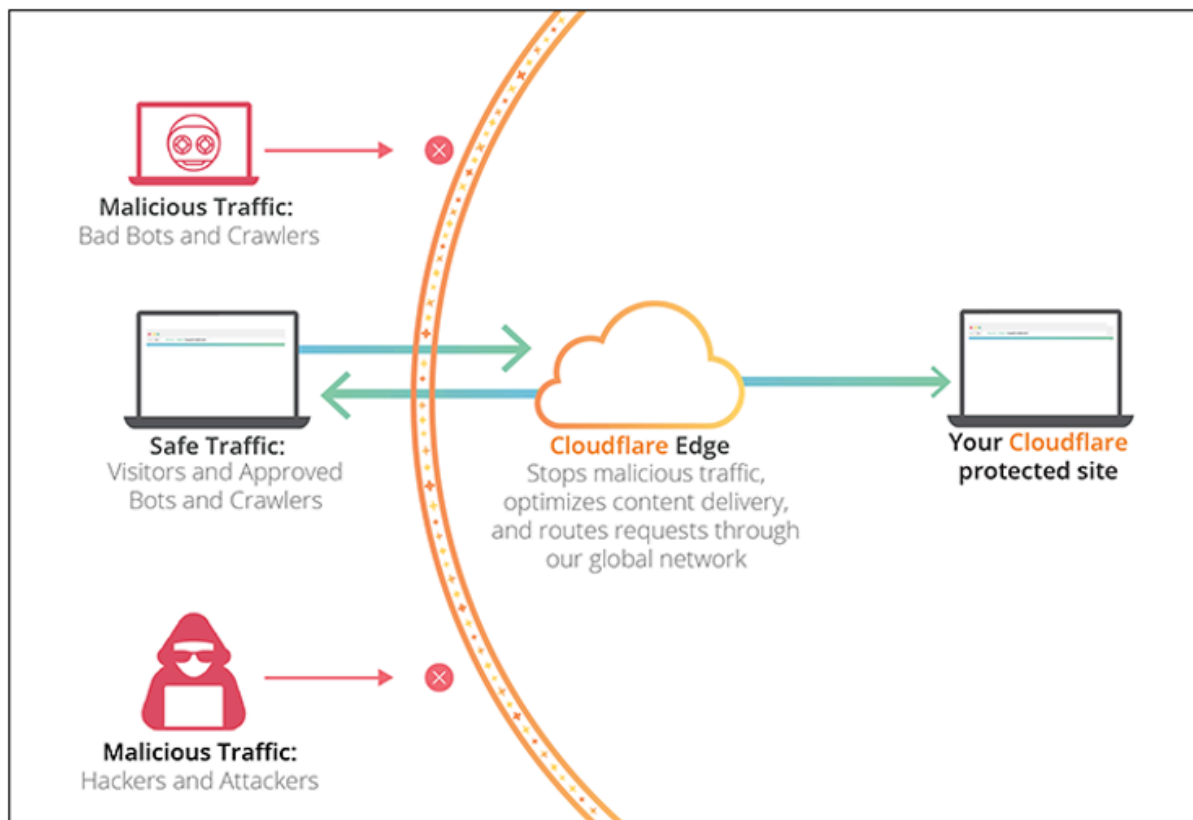
```

```

        if(value !== obj[prop]){
            const prev = obj[prop]
            obj[prop] = value
            callback(`${prop}가 [${prev}] >> [${value}] 로
변경되었습니다`)
        }
        return true
    }
    })
    return proxy
}
const a = {
    "형규" : "솔로"
}
const b = createReactiveObject(a, console.log)
b.형규 = "솔로"
b.형규 = "커플"
// 형규가 [솔로] >> [커플] 로 변경되었습니다

```

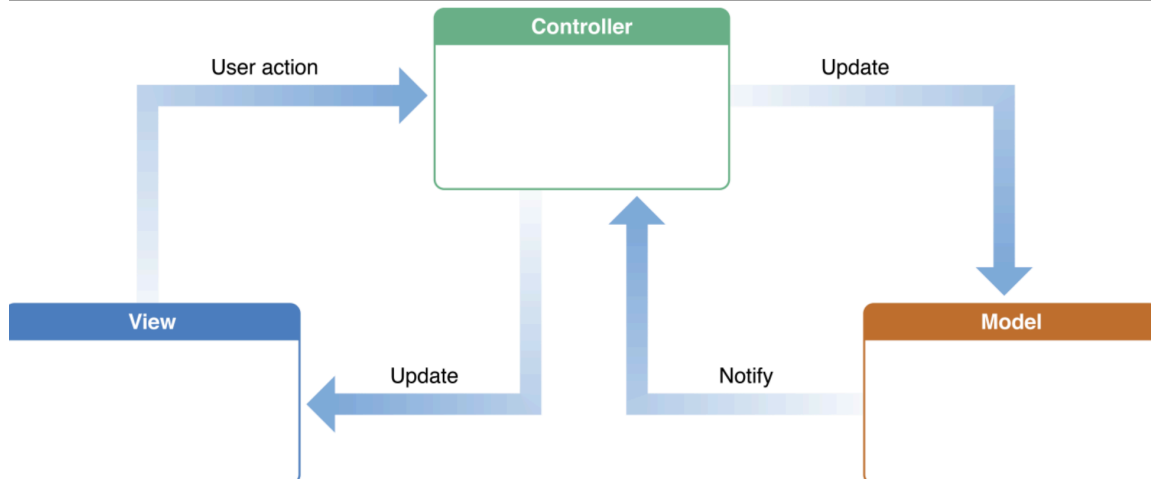
보통은 서비스 앞단에 프록시 서버로 cloudflare를 뒤서 불필요한, 또는 공격적인 트래픽을 막습니다.



MVC MVP MVVM패턴

MVC 패턴

MVC패턴은 모델(Model), 뷰(View), 컨트롤러(Controller)로 이루어진 디자인 패턴입니다.



모델

모델(model)은 애플리케이션의 데이터인 데이터베이스, 상수, 변수 등을 뜻합니다.

뷰에서 데이터를 생성하거나 수정할 때 컨트롤러를 통해 모델이 생성 또는 업데이트 됩니다.

예를 들어 사용자가 네모박스에 글자를 적는다고 해보죠. 이 때 모델은 네모박스의 크기정보, 글자내용, 글자의 위치, 글자의 포맷 정보 등이 됩니다.

뷰

뷰(view)는 inputbox, checkbox, textarea 등 사용자 인터페이스 요소를 나타내며 모델을 기반으로 사용자가 볼 수 있는 화면을 뜻합니다. 모델이 가지고 있는 정보를 따로 저장하지 않아야 하며 변경이 일어나면 컨트롤러에 이를 전달해야 합니다.

컨트롤러

컨트롤러(controller)는 하나 이상의 모델과 하나 이상의 뷰를 잇는 다리 역할을 하며 이벤트 등 메인 로직을 담당합니다. 또한, 모델과 뷰의 생명주기도 관리하며, 모델이나 뷰의 변경 통지를 받으면 이를 해석하여 각각의 구성 요소에 해당 내용에 대해 알려줍니다.

MVC패턴의 장점

1. 애플리케이션의 구성 요소를 세 가지 역할로 구분하여 개발 프로세스에서 각각의 구성 요소에만 집중해서 개발할 수 있습니다.
2. 재사용성과 확장성이 용이하다는 장점이 있습니다.

MVC패턴의 단점

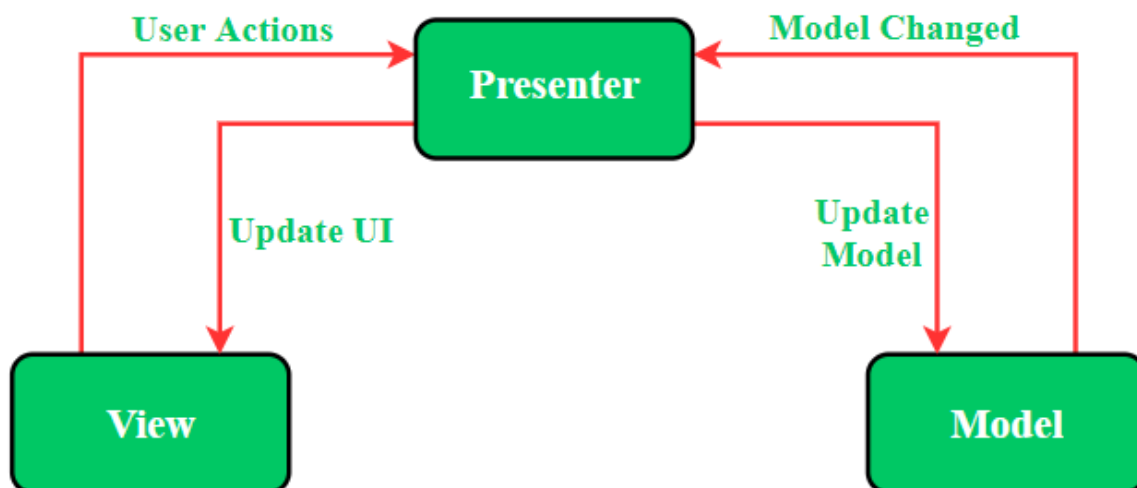
애플리케이션이 복잡해질수록 모델과 뷰의 관계가 복잡해지는 단점이 있습니다.

MVC패턴을 반영한 대표적인 프레임워크로 Spring WEB MVC가 있습니다.



MVP 패턴

C가 P(프레젠테이션, presenter)로 교체된 패턴. V와 P는 1:1 관계이므로 MVC보다 더 강한 결합을 지닌 디자인 패턴입니다.



MVVM 패턴

MVC의 C가 VM(뷰모델, view model)로 바뀐 패턴. VM은 뷰를 추상화한 계층이며 VM : V = 1 : N이라는 관계를 갖습니다.



VM은 커맨드와 데이터바인딩을 가집니다.

- 커맨드 : 여러 요소에 대한 처리를 하나의 액션으로 처리할 수 있는 기법
- 데이터바인딩 : 화면에 보이는 데이터와 브라우저 상의 메모리 데이터를 일치시키는 방법.

MVVM 패턴을 가진 대표적인 프레임워크로는 뷰(Vue.js)가 있습니다.



차이정리

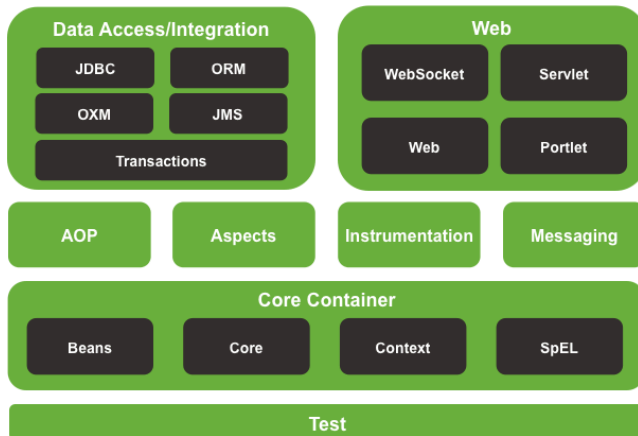
특징	MVC 패턴	MVP 패턴	MVVM 패턴
관계	컨트롤러와 뷰는 1 : n	프레젠테이션과 뷰는 1 : 1	뷰모델과 뷰 는 1 : n
참조	뷰는 컨트롤러를 참조 x	뷰는 프레젠테이션을 참조 o	뷰는 뷰모델을 참조 o

Spring의 MVC패턴 적용사례

Spring은 자바기반으로 애플리케이션 개발을 할 때 많은 기능들을 제공하는 프레임워크입니다. 다음과 같은 아키텍처를 가지죠.



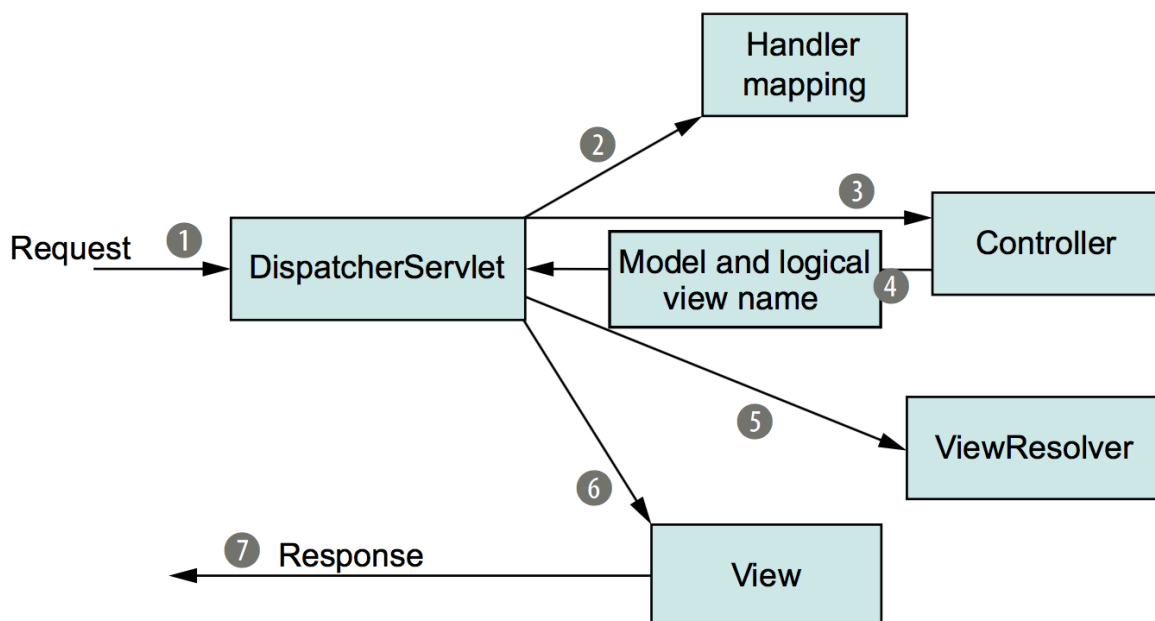
Spring Framework Runtime



이 중에서 MVC패턴이 적용된 Web모듈의 Spring Web MVC를 살펴보겠습니다.

디스패처 서블릿의 요청 처리과정

Spring Web MVC의 MVC패턴은 디스패처 서블릿의 사용자 요청처리과정을 통해 알 수 있습니다.



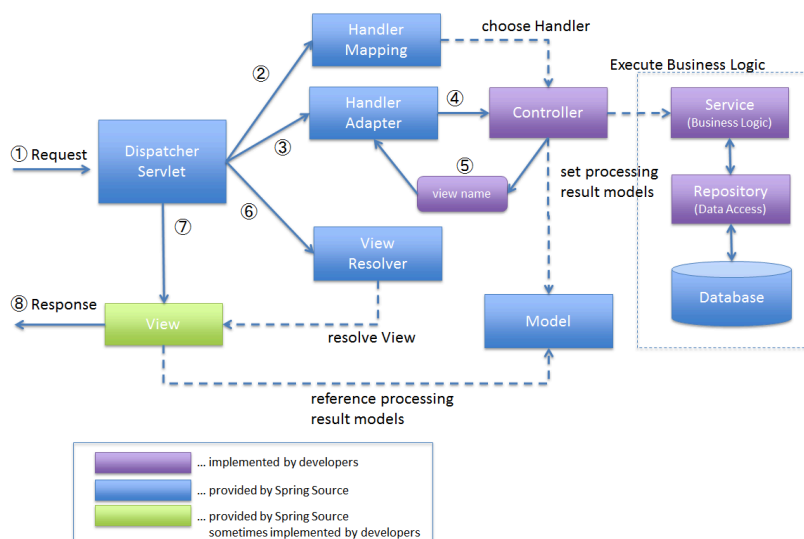
1. 클라이언트가 요청을 했을 때 가장 먼저 디스패처 서블릿이 이를 받습니다.(프론트 컨트롤러 역할) 이 때 url 이나 form data 등 여러개의 데이터가 있겠죠? 이러한 걸 기반으로 어떤 컨트롤러에게 이를 처리하게 결정하는 공장의 반장과도 같은 역할을

합니다. 보통 클래스이름, url, xml의 설정 등을 참고 할 수 있지만 보통 @requestmapping를 참고해서 합니다.

```
@RequestMapping(value = "/ex/foos", method = POST)
@ResponseBody
public String postFoos() {
    return "Post some Foos";
}
```

2. 하나 이상의 handler mapping을 참고해서 적절한 컨트롤러를 설정합니다. 이후 컨트롤러로 요청을 보냅니다.
3. 컨트롤러는 데이터베이스에 접근하여 데이터를 가져오는 등 비즈니스로직을 수행합니다.
4. 그렇게 해서 사용자에게 전달해야할 정보인 모델을 생성합니다.
5. 그 다음 뷰를 구현하기 위한 view resolver를 참고합니다.
6. 해당 정보를 기반으로 뷰를 렌더링합니다.
7. 응답 데이터를 보냅니다.

다시 한번 볼까요? 조금 더 자세한 그림입니다.

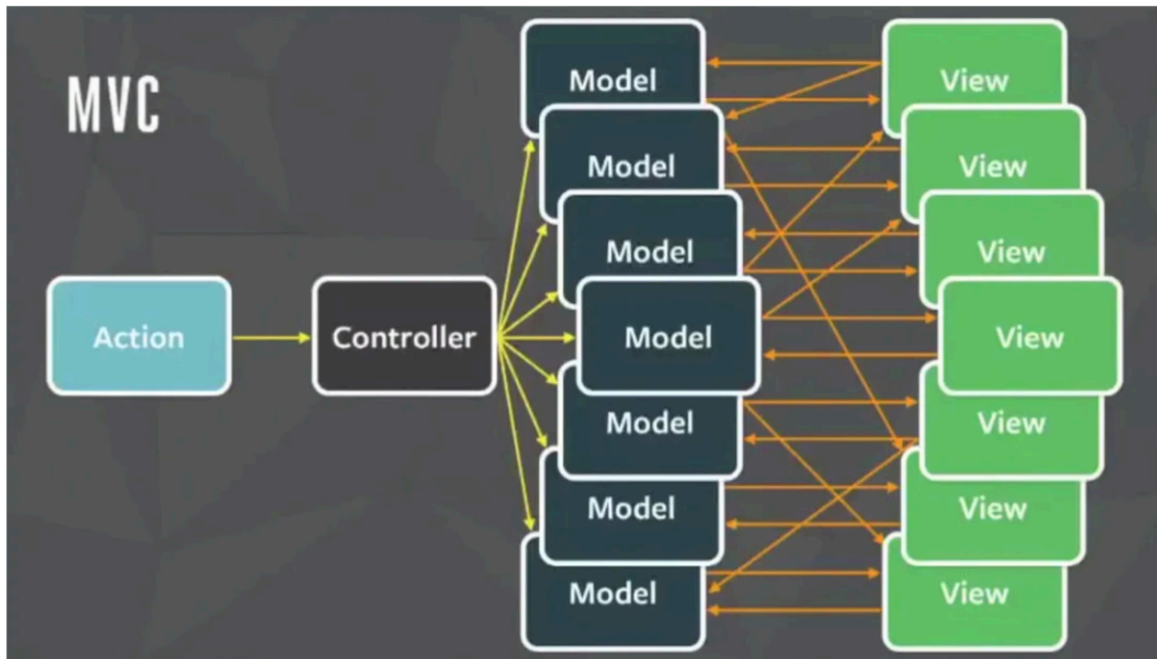


flux패턴

flux패턴은 단방향으로 데이터 흐름을 관리하는 디자인패턴입니다.

ex)

페이스북은 “읽은 표시(mark seen)”에 대한 기능장애를 겪었습니다. 어떤 페이지에서 메시지를 읽었는데 다른 페이지에서는 메시지가 안 읽었다고 뜨는 것이죠. 이는 모델(M, model)과 뷰(view)의 관계가 복잡해지니 버그를 수정하기도 데이터흐름을 알아보기 어려운 문제였습니다.

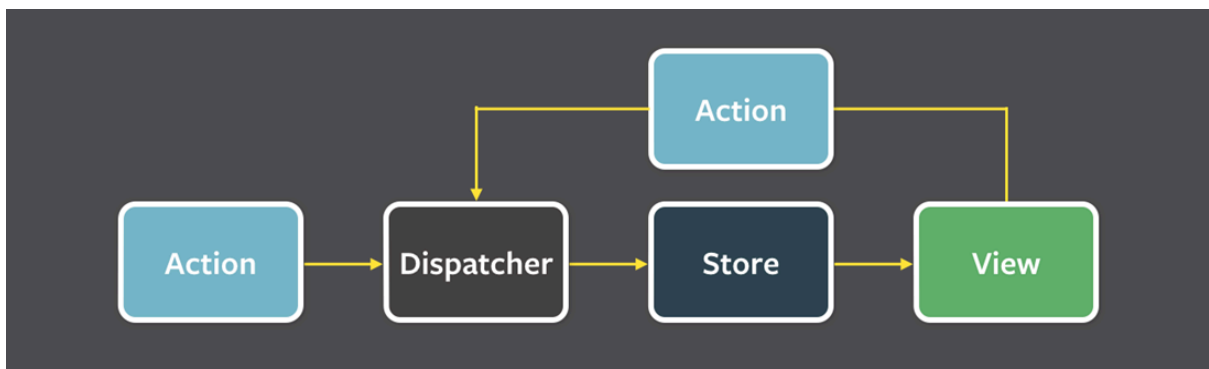


즉, 뷰에서 일어난 것이 모델에 영향을 끼치기도 그 반대로 영향을 미치는 로직도 있는 상황이 발생하여 데이터를 일관성 있게 뷰에 공유하기가 어려웠습니다.

이를 위한 해결방법으로 데이터가 “한방향”으로만 흐르게 flux 패턴이 등장했습니다.

flux패턴은 action, dispatcher, store, view 라는 계층으로 구성됩니다.

flux패턴의 구조



Action

사용자의 이벤트를 담당합니다. 마우스 클릭이나, 글을 쓴다던가 등을 의미하며 해당 이벤트에 관한 객체를 만들어내 dispatcher에게 전달합니다.

Dispatcher

Dispatcher는 들어오는 Action 객체 정보를 기반 어떠한 “행위”를 할 것인가를 결정합니다. 보통 action객체의 type를 기반으로 미리 만들어 놓은 로직을 수행하고 이를 Store에 전달합니다.

Store

스토어는 애플리케이션 상태를 관리하고 저장하는 계층입니다. 도메인의 상태, 사용자의 인터페이스 등의 상태를 모두 저장합니다.

view

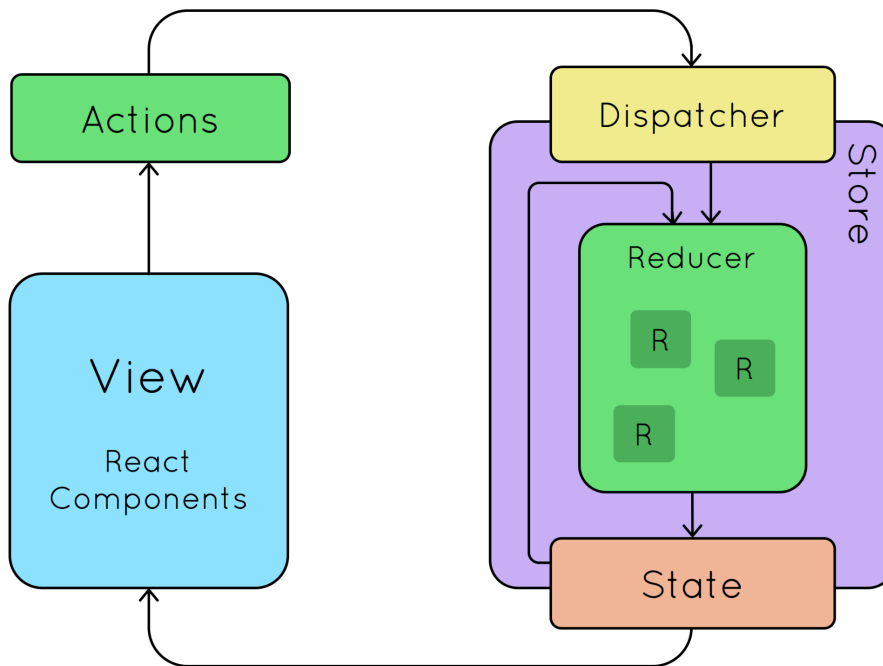
데이터를 기반으로 표출이 되는 사용자인터페이스입니다.

flux 패턴의 장점

flux패턴의 장점으로서는 다음과 같습니다.

- 데이터 일관성의 증대
- 버그를 찾기가 쉬워짐.
- 단위테스팅이 쉬워짐.

flux패턴이 적용된 redux



```
const initialState = {
  visibilityFilter: 'SHOW_ALL',
  todos: []
}

function appReducer(state = initialState, action) {
  switch (action.type) {
    case 'SET_VISIBILITY_FILTER': {
      return Object.assign({}, state, {
        visibilityFilter: action.filter
      })
    }
    case 'ADD_TODO': {
      return Object.assign({}, state, {
        todos: state.todos.concat({
          id: action.id,
          text: action.text,
          completed: false
        })
      })
    }
    case 'TOGGLE_TODO': {
      return Object.assign({}, state, {
        todos: state.todos.map(todo => {
```

```

        if (todo.id !== action.id) {
          return todo
        }

        return Object.assign({}, todo, {
          completed: !todo.completed
        })
      })
    })
  }
  case 'EDIT_TODO': {
    return Object.assign({}, state, {
      todos: state.todos.map(todo => {
        if (todo.id !== action.id) {
          return todo
        }

        return Object.assign({}, todo, {
          text: action.text
        })
      })
    })
  }
  default:
    return state
}
}

```

앞의 코드처럼 switch를 통해 action의 type을 통해 판단 각각의 행위를 결정하는 것을 볼 수 있습니다.

Q. 전략패턴과 의존성주입의 차이는 무엇인가요?

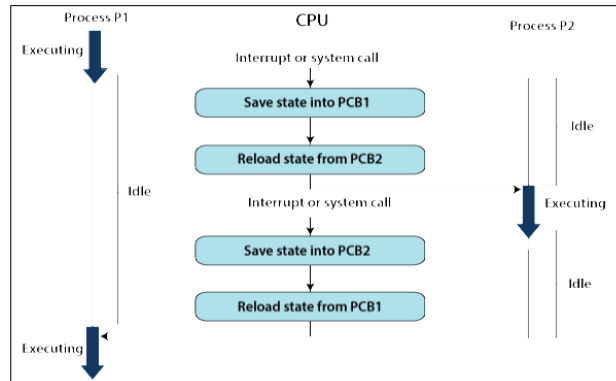
전략패턴, 의존성주입 모두 무언가를 쉽게 교체하기 위한 디자인패턴이라는 공통점이 있습니다. 그러나 다음과 같은 차이가 있습니다.

- 전략패턴: 어떠한 동일한 행동 계약을 기반으로 다양한 구현이 명시되어있는 인터페이스를 만드는 것을 포함합니다.
- 의존성주입 : 단지 일부 동작을 구현하고 의존성을 주입하기만 하는 패턴입니다.

Q. 컨텍스트란 무엇인가요?

컨텍스트는 2가지 뜻으로 씁니다.

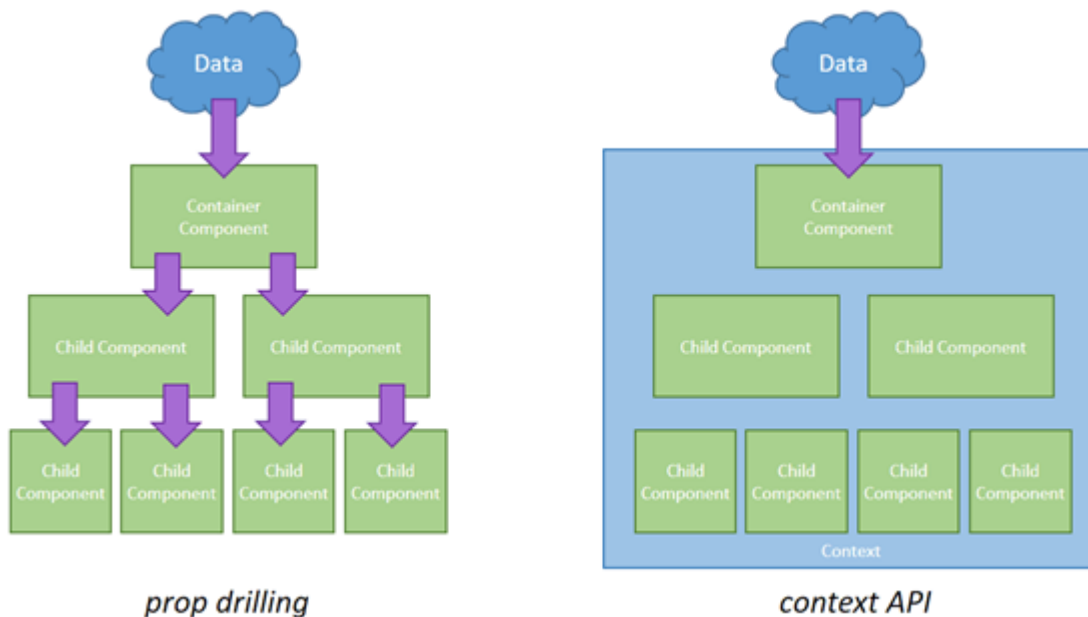
- 1)어떤 종류의 상태, 환경을 캡슐화한 것을 말함.
- 2)작업d이 중단 되고 나중에 같은 지점에서 계속 될 수 있도록 저장하는 최소 데이터 집합.(`context`) 컨텍스트스위칭)



컨텍스트는 context와 contextual information으로 크게 나뉘지는데 다음과 같습니다.

1. 병원에 가면 “이름”과 주민등록번호 앞자리를 말해야죠? 자 여기서 “병원에 방문”하는 context에서 여러분의 이름은 contextual information이 됩니다.
2. HTTP요청을 하는 context에서 HTTP Header는 contextual information이라고 할 수 있습니다.

이러한 이론을 기반으로 한 다양한 모듈 또는 API로는 react.js에서는 전역적으로 “상태”값을 넘길 수 있는 context API가 있습니다.



```

const ThemeContext = React.createContext('light');

class App extends React.Component {
  render() {
    return (
      <ThemeContext.Provider value="dark">
        <Toolbar />
      </ThemeContext.Provider>
    );
  }
}

function Toolbar() {
  return (
    <div>
      <ThemedButton />
    </div>
  );
}

class ThemedButton extends React.Component {
  static contextType = ThemeContext;
  render() {
    return <Button theme={this.context} />;
  }
}

```

React에서 context API를 통해 전역적으로 static contextType을 통해 상태관리를 하는 모습입니다.

여기까지 오시느라 수고하셨습니다.
 질문 있으시면 언제든지 질문 부탁드립니다.
 좋은 수강평과 별점 5점은 제가 큰 힘이 됩니다. :)
 감사합니다.